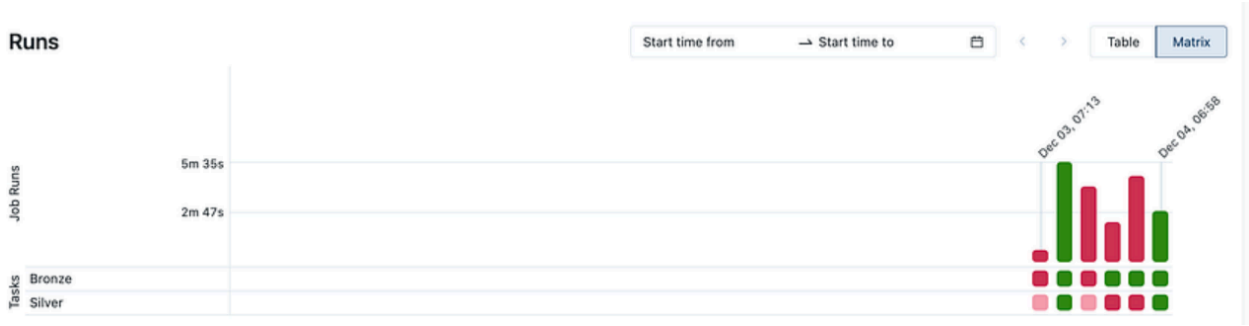
## 🚀 Data Engineering Daily – Day 10

## Page 1/4: The Challenge — Failing PySpark Job




❌ **Scenario:**
A PySpark job processing **billions of rows** keeps failing due to **out-of-memory errors and long shuffle stages**.

**What the interviewer wants to know:**

- Can you identify **why Spark jobs fail at scale?**
- Can you apply **optimization techniques** (not just theory)?
- Can you balance **performance vs cost**?

👉 Common Bottlenecks:

- **Data Skew** (uneven key distribution)
- **Shuffles** (wide transformations like joins/groupBy)
- **Improper partitioning**
- **Broadcasting big tables** instead of small ones

*Swipe → Let's look at an optimization playbook.*

## Page 2/4: The Spark Optimization Playbook




### 1️⃣ Handle Data Skew

- Apply **salting** for skewed join keys.
- Use `spark.sql.adaptive.enabled=true` (AQE) to handle skew dynamically.

### 2️⃣ Optimize Joins

- Use **broadcast joins** for small dimension tables.
- Repartition data on join keys before shuffle-heavy joins.

### 3️⃣ Partitioning & Caching

- Tune `spark.sql.shuffle.partitions` (default = 200, often too high/low).
- Repartition/coalesce based on data size.
- Cache intermediate DataFrames if reused multiple times.

### 4️⃣ Resource Tuning

- Adjust executor memory & cores.
- Use autoscaling clusters in Databricks.

👉 Next → Let's see an example from a real project.

## Page 3/4: Example from a Real Project

```
# Bad Join (caused shuffle skew & OOM)
result = big_df.join(small_df, "customer_id")

# Optimized Join
from pyspark.sql.functions import col, concat_ws, rand

# Add salting to skewed key
salted_big = big_df.withColumn("salt", (rand() * 5).cast("int"))
salted_small = (small_df.crossJoin(spark.range(5))
                .withColumn("salt", col("id")))

# Join with salt
result = salted_big.join(
    salted_small,
    on=["customer_id", "salt"],
    how="inner"
)
```

⚡ **Outcome:**

- Distributed load across tasks.
- Reduced shuffle size.
- Job ran **3x faster** and **no OOM errors**.

👉 Final Verdict →

## Page 4/4: Key Takeaways

### ✅ How to Tackle PySpark Failures in Interviews

- Identify **root cause**: skew, shuffle, partitions, resources.
- Show **optimizations you've applied** (salting, AQE, broadcast joins).
- Emphasize **real-world experience** with examples.

⚖️ **Interview Tip:**
Don't just say *"increase cluster size"*. Show that you understand **how Spark works under the hood**.

👉 **Final Answer (Interview Style):**
"In one project, my Spark job was failing due to data skew on `customer_id`. I applied salting, used AQE for skew handling, and optimized joins with broadcast where applicable. This reduced shuffle, eliminated OOM errors, and cut runtime by 70%."

💡 Tomorrow (Day 11): "How to Design a Fault-Tolerant Data Pipeline in ADF."